

# Unions in C

# Unions in C

Similar to Structures - It is a derived data-type to organize a group of related data items of different data types referring to a single entity

i.e. a single variable capable of holding data items of different data types

In structure, each member has its own storage location, whereas all the members of the union share a same location.

The compiler allocates a piece of storage that is large enough to hold the largest variable type in the union.

Because of this kind of storage, union it can handle only one member at a time

# Unions in C

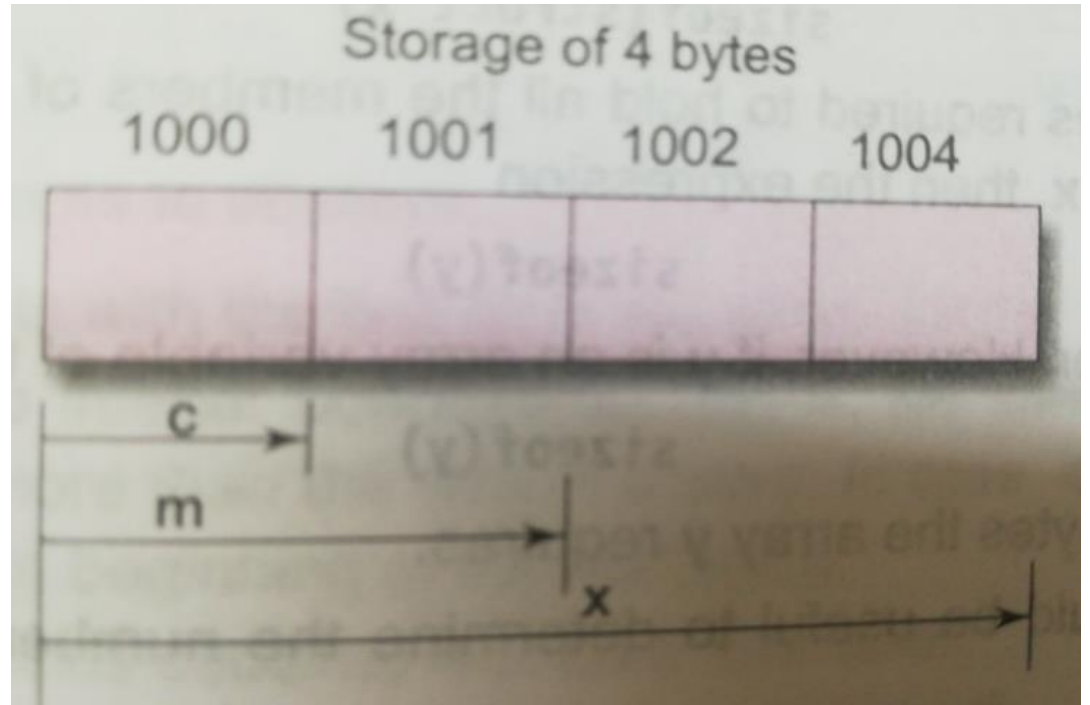
## Example:

```
union item
{
    int m;
    float x;
    char c;
} code;
```

To access a union member,

code.m

code.x



# Unions in C

**For example,**

```
code.m=200;  
code.x=22.33;  
printf(“%d”, code.m);    // error  
printf(“%f”, code.x);    // valid
```

Note: you can access the value that is recently stored. i.e. the new value will supersedes the previous value.

Union can be used in all the places where structures can be used.

# Unions in C

Union may be initialized when the variable is declared.

It can be initialized only with a value of the same type.

For example,

```
union item abc = {100};           //valid
```

```
union item abc = {10.20};        //invalid
```

Other members can be initialized by either assigning values or reading from keyboard

# Unions in C

```
#include <stdio.h>
union sample
{
    int regno;
    float mark;
}s1;

int main()
{
    printf("Enter the register number: ");
    scanf("%d",&s1.regno);
    printf("\n Entered register number: %d",s1.regno);
    s1.mark=100;
    printf("\n Mark: %f",s1.mark);
    return 0;
}
```

## Output

**Enter the register number: 121**  
**Entered register number: 121**  
**Mark: 100.000000**

# Structure vs Union

| Structure                                                                                                                                                                                                                                     | Union                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.The keyword <b>struct</b> is used to define a structure                                                                                                                                                                                     | 1. The keyword union is used to define a union.                                                                                                                                           |
| 2. When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is greater than or equal to the sum of sizes of its members. The smaller members may end with unused slack bytes. | 2. When a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of union is equal to the size of largest member. |
| 3. Each member within a structure is assigned unique storage area of location.                                                                                                                                                                | 3. Memory allocated is shared by individual members of union.                                                                                                                             |
| 4. The address of each member will be in ascending order This indicates that memory for each member will start at different offset values.                                                                                                    | 4. The address is same for all the members of a union. This indicates that every member begins at the same offset value.                                                                  |
| 5 Altering the value of a member will not affect other members of the structure.                                                                                                                                                              | 5. Altering the value of any of the member will alter other member values.                                                                                                                |
| 6. Individual member can be accessed at a time                                                                                                                                                                                                | 6. Only one member can be accessed at a time.                                                                                                                                             |
| 7. Several members of a structure can initialize at once.                                                                                                                                                                                     | 7. Only the first member of a union can be initialized.                                                                                                                                   |